



# Documento de Requisitos do Produto (PRD)

## DropZone — Sistema de Gestão de Achados e Perdidos

Status: pronto para apresentação / produção inicial

Data: 03/06/2026

PM/Autor: Jhonathan Neves de Sousa

### 1. Escolha do Sistema

#### Sistema escolhido

- DropZone: gestão de achados e perdidos.
- Produto web com frontend, backend e banco remoto.
- Foco em rastreabilidade, segurança e devolução correta.
- Aplicável a ambiente acadêmico/institucional.

### 2. Contexto e Problema

#### Dor e solução

- Antes: controle manual, informal e descentralizado.
- Usuários não sabiam onde procurar itens.
- Risco de entrega errada por falta de validação.
- Agora: busca, código único, auditoria e chat P2P.

### 3. Stakeholders

#### Partes interessadas

- Usuários base: alunos/servidores.
- Administradores do setor responsável.
- Superusuário do sistema.
- Instituição/campus.
- Equipe técnica e professor avaliador.

### 4. Necessidades

#### Necessidades dos usuários

- Consultar itens rapidamente.
- Registrar perda com fotos e detalhes.
- Solicitar coleta com segurança.
- Acompanhar minhas solicitações.
- Conversar com quem encontrou o item.
- Confirmar devoluções com rastreabilidade.

### 5. Requisitos Funcionais

#### Escopo funcional

- Login, cadastro e perfil.
- Home com vitrine, filtros e detalhes.
- Cadastro/edição de itens por admin/super.
- Perdi um item e minhas solicitações.
- Código único de coleta.
- Gestão de usuários, auditoria e P2P.

### 6. Regras de Negócio

#### Regras principais

- Usuário base não cadastra/edita item encontrado.
- Admin confirma coleta pelo código.
- Super cria admin e promove superusuário.
- Código é único e de uso único.
- Item devolvido não pode ser coletado novamente.
- Logs são somente leitura.

### 7. Requisitos Não Funcionais

#### Qualidade técnica

- Frontend: Next.js, React, TypeScript, Tailwind.
- Backend: Node.js, Express, PostgreSQL/Supabase.
- API REST com Axios no frontend.
- Deploy: Vercel + Render.
- Interface responsiva para desktop e mobile.
- Loading states, toasts e feedback visual.

### 8. MVP e Mobile

#### MVP entregue

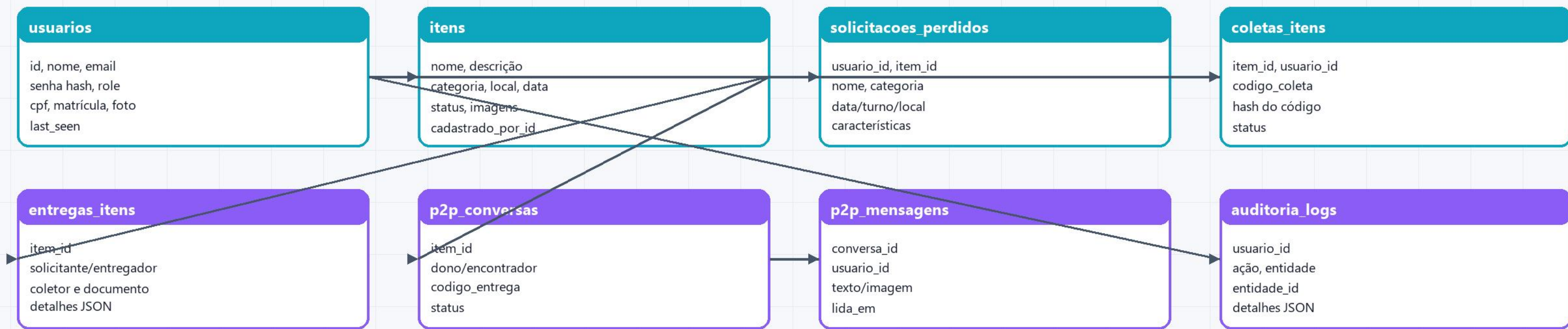
- Login/cadastro funcionando.
- Vitrine de itens com busca/filtros.
- Cadastro de item perdido/encontrado.
- Código de coleta e confirmação.
- Gestão de usuários e auditoria.
- Chat P2P e relatórios.
- Roda em navegador mobile.

### 9. Segurança

#### Controles implementados

- JWT para autenticação.
- bcrypt para senhas e códigos.
- Controle por perfil: user/admin/super.
- RLS habilitado no Supabase.
- CPF mascarado no perfil.
- Logout automático por inatividade.
- Uploads validados.

### 10. Modelo de Dados



### 11. Evidências de Entrega

#### Como comprovar

- Link frontend: sigap-front.vercel.app.
- Link backend: Render /api/health.
- Print desktop: login, home e admin.
- Print mobile: login, home e perdi um item.
- PDF/PNG deste PRD anexado à entrega.

### 12. Próximos Passos

#### Futuro

- Notificações por email/push.
- WebSocket para chat em tempo real.
- Dashboard com gráficos.
- Supabase Storage/S3 para imagens.
- Testes E2E automatizados.
- PWA ou app mobile.

### 13. Checklist Final

#### Itens da atividade

- 1. Escolha do sistema: OK.
- 2. Stakeholders: OK.
- 3. Necessidades dos usuários: OK.
- 4. RF/RNF/Regras: OK.
- 5. Modelo de dados: OK.
- 6. MVP mobile: OK.
- 7. Documentação PRD: OK.